

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : To Study and Implement 8 Queens Problem			
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/08		ISSUE NO. : 00	ISSUE DATE : 01.07.2023	
REV. DATE :		REV. NO. :	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : Design and Analysis of Algorithm			SEMESTER : VI	PAGE: 1 OF 5

1.0) AIM:

To Study and Implement 8 Queens Problem

2.0) SCOPE:

- To implement 8 Queens Problem using C Programming.

3.0) FACILITIES/ APPARATUS:

i) **Software:** Turbo C

4.0) THEORY:

The eight queens problem is the problem of placing eight queens on an 8×8 chessboard such that none of them attack one another (no two are in the same row, column, or diagonal). More generally, the n queens problem places n queens on an n×n chessboard.

Explanation:

- This **pseudocode uses a backtracking algorithm** to find a solution to the 8 Queen problem, which consists of placing 8 queens on a chessboard in such a way that no two queens threaten each other.
- The algorithm starts by placing a queen on the first column, then it proceeds to the next column and places a queen in the **first safe** row of that column.
- If the algorithm reaches the 8th column and **all queens are placed in a safe position**, it prints the board and returns true.
If the algorithm is unable to place a queen in a safe position in a certain column, it backtracks to the previous column and tries a different row.
- The “isSafe” function **checks** if it is safe to place a queen on a certain row and column by checking if there are any queens in the same row, diagonal or anti-diagonal.
- It’s worth to notice that this is just a high-level **pseudocode** and it might need to be adapted depending on the specific implementation and language you are using.

Introduction

Backtracking is a recursive approach for solving any problem where we must search among all the possible solutions following some constraints. More precisely, we can say that it is an improvement over the brute force technique. In this blog, we will learn one popular DSA question: 8 queens problem using Backtracking.

We will retain the backtracking approach and go through its time optimization. Let's start with the problem statement of solving 8 queens problem using backtracking in the next section.

Problem Statement

Given an 8x8 chess board, you must place 8 queens on the board so that no two queens attack

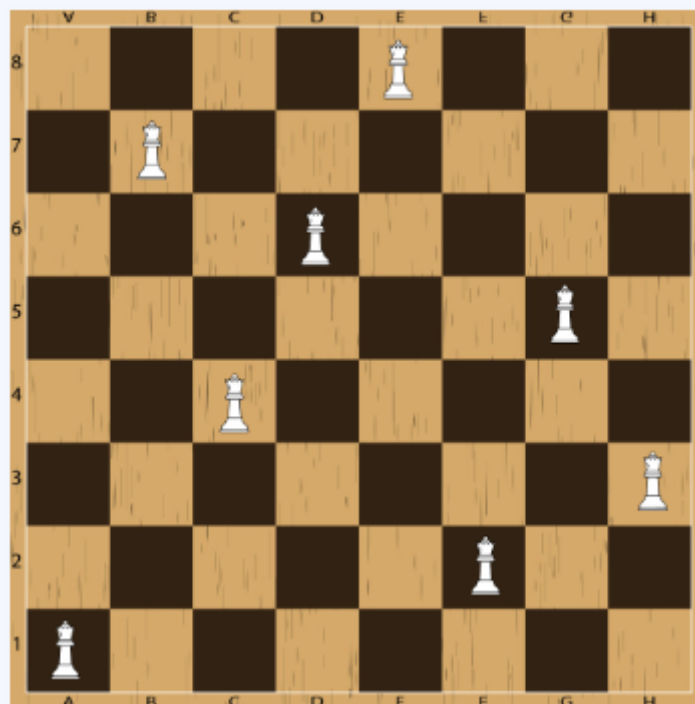
EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/08	ISSUE NO. : 00	ISSUE DATE : 01.07.2023
--	----------------	-------------------------

each other. Print all possible matrices satisfying the conditions with positions with queens marked with '1' and empty spaces with '0'. You must solve the 8 queens problem using backtracking.

- **Note 1:** A queen can move vertically, horizontally and diagonally in any number of steps.
- **Note 2:** You can also go through the [N-Queen Problem](#) for the general approach to solving this problem.

Sample Example

Example: One possible solution to the 8 queens problem using backtracking is shown below. In the first row, the queen is at E8 square, so we have to make sure no queen is in column E and row 8 and also along its diagonals. Similarly, for the second row, the queen is on the B7 square, thus, we have to secure its horizontal, vertical, and diagonal squares. The same pattern is followed for the rest of the queens.



Output:

```
0 0 0 0 1 0 0 0
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
1 0 0 0 0 0 0 0
```

Bruteforce Approach

One brute-force approach to solving this problem is as follows:

- Generate all possible permutations of the numbers 1 to 8, representing the columns on the chessboard.
- For each permutation, check if it represents a valid solution by checking that no two queens are in the same row or diagonal.
- If a valid solution is found, print the board layout.

While this approach works for small numbers, it quickly becomes inefficient for larger sizes as the number of permutations to check grows exponentially. More efficient algorithms, such as backtracking or genetic algorithms, can be used to solve the problem in a more optimized way.

Backtracking Approach

This approach rejects all further moves if the solution is declined at any step, goes back to the previous step and explores other options.

Algorithm

Let's go through the steps below to understand how this algorithm of solving the 8 queens problem using backtracking works:

- **Step 1:** Traverse all the rows in one column at a time and try to place the queen in that position.
- **Step 2:** After coming to a new square in the left column, traverse to its left horizontal direction to see if any queen is already placed in that row or not. If a queen is found, then move to other rows to search for a possible position for the queen.
- **Step 3:** Like step 2, check the upper and lower left diagonals. We do not check the right side because it's impossible to find a queen on that side of the board yet.
- **Step 4:** If the process succeeds, i.e. a queen is not found, mark the position as '1' and move ahead.
- **Step 5:** Recursively use the above-listed steps to reach the last column. Print the solution matrix if a queen is successfully placed in the last column.
- **Step 6:** Backtrack to find other solutions after printing one possible solution.

```
#include<stdio.h>
```

```
#include<math.h>
```

```
void queen(int row, int p);
```

```
int chess[8],count;
```

```
int main()/*from w w w . j a v a 2 s . c o m*/
```

```
{
```

EXPERIMENT NO. : SSGMCE/WI/IT/01/6IT07/08	ISSUE NO. : 00	ISSUE DATE : 01.07.2023
LABORATORY : Design and Analysis of Algorithm Lab (6IT07)	SEMESTER : VI	PAGE: 4 OF 5

```

int p = 8;
    queen(1,p);
    return 0;
}

void print(int p)
{
    int i,j;
    printf("\n\nThis is Solution no. %d:\n\n",++count);
    for(i=1;i<=p;++i)
        printf("\t%d",i);

    for(i=1;i<=p;++i){
        printf("\n\n%d",i);
        for(j=1;j<=p;++j){
            if(chess[i]==j)
                printf("\tQ");
            else
                printf("\t-");
        }
    }

    printf("\n\nThere are total 92 solutions for 8-queens problem.");
}

int place(int row,int column)
{
    int i;
    for(i=1;i<=row-1;++i)

```

```
{
    if(chess[i]==column)
        return 0;
    else
        if(abs(chess[i]-column)==abs(i-row))
            return 0;
}

return 1;
}

void queen(int row,int p)
{
    int column;
    for(column=1;column<=p;++column)
    {
        if(place(row,column))
        {
            chess[row]=column;
            if(row==p)
                print(p);
            else
                queen(row+1,p);
        }
    }
}
```

5.0) Conclusion:

Implemented 8 Queens Problem using backtracking approach in C Programming.

PREPARED BY:
PROF.A.G.SHARMA

APPROVED BY: (H.O.D.)
DR. A. S. MANEKAR